P2505R5 Monadic Functions for `std::expected`

Authors: Jeff Garland

Audience: LEWG, LWG

Project: ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Contact: jeff@crystalclearsoftware.com

Introduction

With the final plenary vote of P0798 Monadic Functions for std::optional complete, we now have an design inconsistency with `std::expected`. P0323 std::expected has now also been voted into the working draft for C++23. This proposal corrects the inconsistency by adding 4 functions to `std::expected` and is targeted at C++23. The author believes this should be treated as a consistency/bug fix still in scope for C++23.

The intent from P0798 clearly states that `std::expected` should gain the same functions:

> *There is a proposal for std::expected which would benefit from many of these same ideas. If the idea to add monadic interfaces to standard library classes on a case-by-case basis is chosen rather than a unified non-member function interface, then compatibility between this proposal and the std::expected one should be maintained*

Motivation and Scope

The following 3 functions are added to `std::optional`, but are currently not part of `std::expected`.

- `and_then` compose a chain of functions returning an expected
- `or_else` returns if it has a value, otherwise it calls a function with the error type
- `transform` apply a function to change the value (and possibly the type)

After feedback, this draft also proposes the addition of two additional functions for `expected` to facilitate additional cases:

- `transform_error` apply a function to change the value, otherwise call a function with error type
- `error_or` a value to return when there is not an error

For example, given the following:

```
using time_expected = expected<boost::posix_time::ptime, std::string>;

time_expected from_iso_str( std::string time )
{
  try {
    ptime t  = boost::posix_time::from_iso_string( time );
    return t;
  }
  catch( std::exception& e )
  {
     return unexpected( e.what() + " str was: "s + time);
  }
}

// for use with transform
ptime next_day( boost::posix_time::ptime t )
{
   return t + boost::gregorian::days(1);
}

// for use with or_else
void print_error( std::string error )
{
   cout << error << endl;
}


//valid iso string
const std::string ts ( "20210726T000000" );
```

Before the change we'd write this:

Before

```
time_expected d = from_iso_str( ts );
if (d)
{
   ptime t = next_day( *d );
}
else
{
   print_error( d.error() );
}
```

And after, this:

After

```
// chain a series of functions until there's an error
auto d = from_iso_str ( ts )
               .or_else( print_error )
               .transform( next_day ) ;
```

Jonathan Wakely has a live demo illustrating the api here https://godbolt.org/z/3a1j6d63a. For more examples see: https://godbolt.org/z/K7rbhx9oo.

Design Considerations

`expected<void, E>`

Compared with `optional`, `expected` provides the capability for `void` as a function return type. This complicates the design compared to `optional`. As currently specified the `void` specialization of `expected`, `expected<void, T>` removes the `value_or` function since `void` cannot be initialized to anything else.

While the primary implementation for this paper excludes `transform` for `expected<void, E>` and the original version removed this overload, feedback from reviewers suggested the overload is valuable. Thus the new version allows for this overload.

`transform` to `expected<void, E>` return

Consider the following code.

```
struct E{};
auto x = expected<int, E>(4).transform([](int){});
```

The original specifications and implementations would fail to compile this code. A reasonable alternative is to return `expected<void, E>`. The specification has been changed to allow this behavior.

`transform_error` function

The R0 version of this paper did not include this function to match optional, but several reviewers noted the important utility of this case. Note that in the reference implementation of this paper, the function is called `map_error`. The R1 version of the paper names the function `transform_error` after discussions with various reviewers.

Changing of error return types

One complication of the monadic functions for `expected` as compared to `optional` is the ability to change the return error return type on subsequent function calls. Optional, of course, can only return `std::nullopt_t`.

Thus we would like to support use cases like the following:

```
std::expected<SomeUserType, InternalError> initialize();

std::expected<SomeUserType, UserError>
makeUserError(InternalError err)
{
   return std::unexpected( UserError(err) );
};
```

```
std::expected<SomeUserType, UserError> result = initialize().or_else( m
```

Note that the changing of the error type applies to the `or_else` and `transform_error` overloads only.

Use of deducing-this in the specification

This question has been asked several times including in the LEWG 2022-05-03 meeting. While less concise, the author prefers to keep the specification consistent with the rest of the c++23 library and especially `std::optional`. Limited access to compilers that implement deducing-this make it difficult to confirm correct behavior of a deducing-this based specification. In addition, the current specification does not preclude changing to a deducing-this based consistently with other library facilities in a future release.

Adding `error_or` functions

It has been suggested that `error_or` functions be added to the proposal. While the author is sympathetic to the suggestion, since consistency with `optional` for c++23 is the intent these are not proposed. These can be added later or if LEWG would prefer the author can investigate adding them in this cycle.

At 2022-06-07 LEWG telecom poll was taken to add this function. The wording is added in R4 of the paper.

POLL: Add `error_or` to `expected` for C++23.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
4	7	2	2	0

Attendance: 21

Author Position: N

Outcome: Consensus in favor.

As an example usage consider the following code:

Before

```
//before
file create_file(char const*, std::error_code&);
bool test_and_report(const error_code&);

std::error_code ec;
file f = create_file("file", ec);
if (test_and_report(ec)) { return; }
```

After

```
expected<file, error_code> create_file(char const*);
bool test_and_report(const error_code&);

auto f = create_file("file");

if (test_and_report(f.error_or({}))) { return; }
```

Free versus member functions

One design question that has been raised is the use of member functions instead of free functions. The author is sympathetic to this view, but believes this can only be done by changing both `optional` and `expected`. The expressed benefits of free functions would be:

- better compatibility with future library facilities
- more generic code – singular specification

While in theory, free functions provide consistency with some future version of the library where functions such as `then` and `upon_error` from P2300 `std::execution` would be part of the standard library. However, `then` is specified using concepts such as `std::execution::sender` which are not appropriate for a generalized `then` functions for `optional` and `expected`. In fact, attempting to specify this as free functions now might trample on names for future library free functions.

Since the author is unaware of any existing practice with providing these functions and the purpose of the paper is consistency with monadic optional functions already voted into the working draft, the author will not pursue this option. A fully thought out paper with existing practice would be required to make this change.

Note that the author's experience with `std::expected` implementing json parsing is that `expected` would be extremely cumbersome for the task without these functions. So shipping C++23 without these functions for an unclear future benefit would be a major mistake for users.

Other Languages

Rust provides a result type with similar functions monadic functions.

https://doc.rust-lang.org/std/result/enum.Result.html

Implementations

Jonathan Wakely has implemented the full proposal here https://github.com/jwakely/gcc/blob/expected/libstdc%2B%2B-v3/include/std/expected.

The core of the proposal was originally implemented by Sy Brand https://github.com/TartanLlama/expected with documentation here.

Rishabh Dwivedi has also implemented the proposal based on the draft wording https://github.com/RishabhRD/expected.

Wording

Feature Test Macro

Update the `__cpp_lib_expected` feature test macro from P0323.

Class template expected [expected.expected]

After `value_or` functions in synopsis at [expected.object.obs] add the following:

```
template<class G = E> constexpr E error_or(G&&) const &;
template<class G = E> constexpr E error_or(G&&) &&;

// [expected.object.monadic], monadic operations
template <class F> constexpr auto and_then(F&& f) &;
template <class F> constexpr auto and_then(F&& f) &&;
template <class F> constexpr auto and_then(F&& f) const &;
template <class F> constexpr auto and_then(F&& f) const &&;
template <class F> constexpr auto or_else(F&& f) &;
template <class F> constexpr auto or_else(F&& f) &&;
template <class F> constexpr auto or_else(F&& f) const &;
template <class F> constexpr auto or_else(F&& f) const &&;
template <class F> constexpr auto transform(F&& f) &;
template <class F> constexpr auto transform(F&& f) &&;
template <class F> constexpr auto transform(F&& f) const &;
template <class F> constexpr auto transform(F&& f) const &&;
template <class F> constexpr auto transform_error(F&& f) &;
template <class F> constexpr auto transform_error(F&& f) &&;
template <class F> constexpr auto transform_error(F&& f) const &;
template <class F> constexpr auto transform_error(F&& f) const &&;
```

[expected.object.general] changes

Replace paragragh 2 sentence 1 as follows: ~~A program that instantiates the definition of template expected<T, E> for a reference type, a function type, or for possibly cv-qualified types in~~ $_{\text{placet}}$~~, unexpect~~ $_{\text{t}}$~~, or a specialization of unexpected for the T parameter is ill-formed.~~

with

A type T is a *valid value type for* `expected`, if `remove_cv_t<T>` is `void` or a complete non-array object type that is not `in_place_t`, `unexpect_t`, or a specialization of `unexpected`. A program which instantiates class template `expected<T, E>` with an argument T that is not a valid value type for `expected` is ill-formed.

Add following at the end [expected.object.obs] observers:

1 `template <class G = E> constexpr E error_or(G&& e) const &;`

Mandates: `is_copy_constructible_v<E>` is `true` and `is_convertible_v<G, E>` is `true`.

Returns: `std::forward<G>(e)` if `has_value()` is `true`, `error()` otherwise.

2 `template <class G = E> constexpr E error_or(G&& e) &&;`

Mandates: `is_move_constructible_v<E>` is `true` and `is_convertible_v<G, E>` is `true`.

Returns: `std::forward<G>(e)` if `has_value()` is `true`, `std::move(error())` otherwise.

Add a new sub-clause [expected.object.monadic] Monadic operations between [expected.object.obs] and [expected.object.eq]

1 `template <class F> constexpr auto and_then(F&& f) &;`

2 `template <class F> constexpr auto and_then(F&& f) const &;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(value())>>`.

Constraints: `is_copy_constructible_v<E>` is `true`.

Mandates: U is a specialization of `expected` and `is_same_v<U::error_type, E>` is `true`.

Effects: Equivalent to:

```
if (has_value())
   return invoke(std::forward<F>(f), value());
else
   return U(unexpect, error());
```

1 `template <class F> constexpr auto and_then(F&& f) &&;`

2 `template <class F> constexpr auto and_then(F&& f) const &&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(std::move(value()))>>`.

Constraints: `is_move_constructible_v<E>` is `true`.

Mandates: U is a specialization of `expected` and `is_same_v<U::error_type, E>` is `true`.

Effects: Equivalent to:

```
if (has_value())
   return invoke(std::forward<F>(f), std::move(value()));
else
   return U(unexpect, std::move(error()));
```

1 `template <class F> constexpr auto or_else(F&& f) &;`

2 `template <class F> constexpr auto or_else(F&& f) const &;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_copy_constructible_v<T>` is `true`.

Mandates: G is a specialization of `expected` and `is_same_v<G::value_type, T>` is `true`.

Effects: Equivalent to:

```
if (has_value())
    return G(in_place, value());
else
    return invoke(std::forward<F>(), error());
```

1 `template <class F> constexpr auto or_else(F&& f) &&;`

2 `template <class F> constexpr auto or_else(F&& f) const &&;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

Constraints: `is_move_constructible_v<T>` is `true`.

Mandates: G is a specialization of `expected` and `is_same_v<G::value_type, T>` is `true`.

Effects: Equivalent to:

```
if (has_value())
   return G(in_place, std::move(value()));
else
   return invoke(std::forward<F>(f), std::move(error()));
```

1 `template <class F> constexpr auto transform(F&& f) &;`

2 `template <class F> constexpr auto transform(F&& f) const &;`

Let U be `remove_cv_t<invoke_result_t<F, decltype(value())>>`.

Mandates: U is a valid value type for `expected`. If `is_void_v<U>` is `false`, the declaration `U u(invoke(std::forward<F>(f), value()));` is well-formed.

Constraints: `is_copy_constructible_v<E>` is `true`.

Effects:

- If `has_value()` is `false`, returns `expected<U,E>(unexpect, error())`.
- Otherwise, if `is_void_v<U>` is `false`, returns an `expected<U, E>` object whose `has_val` member is `true` and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f), value())`.
- Otherwise, evaluates `invoke(std::forward<F>(f), value())` and then returns `expected<U,E>()`.

1 `template <class F> constexpr auto transform(F&& f) &&;`

2 `template <class F> constexpr auto transform(F&& f) const &&;`

Let U be `remove_cv_t<invoke_result_t<F, decltype(std::move(value()))>>`.

Mandates: U is a valid value type for `expected`. If `is_void_v<U>` is `false`, the declaration `U u(invoke(std::forward<F>(f), std::move(value())));` is well-formed.

Constraints: `is_move_constructible_v<E>` is `true`.

Effects:

- If `has_value()` is `false`, returns `expected<U,E>(unexpect, std::move(error()))`.

- Otherwise, if `is_void_v<U>` is `false`, returns an `expected<U, E>` object whose `has_val` member is `true` and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(value()))`.
- Otherwise, evaluates `invoke(std::forward<F>(f), std::move(value()))` and then returns `expected<U,E>()`.

1 `template <class F> constexpr auto transform_error(F&& f) &;`

2 `template <class F> constexpr auto transform_error(F&& f) const &;`

Let G be `remove_cv_t<invoke_result_t<F, decltype(error())>>`.

Mandates: G is a valid value type for `expected` and the declaration `G g(invoke(std::forward<F>(f), error()));` is well-formed.

Constraints: `is_copy_constructible_v<T>` is `true`.

Returns: If `has_value()` is `true`, `expected<T,G>(in_place, value())`; otherwise an `expected<T, G>` object whose `has_val` member is `false` and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), error())`.

1 `template <class F> constexpr auto transform_error(F&& f) &&;`

2 `template <class F> constexpr auto transform_error(F&& f) const &&;`

Let G be `remove_cv_t<invoke_result_t<F, decltype(std::move(error()))>>`.

Mandates: G is a valid value type for `expected` and the declaration `G g(invoke(std::forward<F>(f), std::move(error())));` is well-formed.

Constraints: `is_move_constructible_v<T>` is `true`.

Returns: If `has_value()` is `true`, `expected<T,G>(in_place, std::move(value()))`; otherwise, an `expected<T, G>` object whose `has_val` member is `false` and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(error()))`.

Partial specialization of expected for void types [expected.void.general]

After `error` functions [expected.void.obs] section add:

```
template<class G = E> constexpr E error_or(G&&) const &;
template<class G = E> constexpr E error_or(G&&) &&;

// [expected.void.monadic], monadic
template <class F> constexpr auto and_then(F&& f) &;
template <class F> constexpr auto and_then(F&& f) &&;
template <class F> constexpr auto and_then(F&& f) const &;
template <class F> constexpr auto and_then(F&& f) const &&;
template <class F> constexpr auto or_else(F&& f) &;
template <class F> constexpr auto or_else(F&& f) &&;
template <class F> constexpr auto or_else(F&& f) const &;
template <class F> constexpr auto or_else(F&& f) const &&;
template <class F> constexpr auto transform(F&& f) &;
template <class F> constexpr auto transform(F&& f) &&;
template <class F> constexpr auto transform(F&& f) const &;
template <class F> constexpr auto transform(F&& f) const &&;
template <class F> constexpr auto transform_error(F&& f) &;
template <class F> constexpr auto transform_error(F&& f) &&;
template <class F> constexpr auto transform_error(F&& f) const &;
template <class F> constexpr auto transform_error(F&& f) const &&;
```

Partial specialization of expected for void types [expected.void.obs]

At the end of Observers section after `error` functions add:

1 `template <class G = E> constexpr E error_or(G&& e) const &;`

Mandates: `is_copy_constructible_v<E>` is `true` and `is_convertible_v<G, E>` is `true`.

Returns: `std::forward<G>(e)` if `has_value()` is `true`, `error()` otherwise.

2 `template <class G = E> constexpr E error_or(G&& e) &&;`

Mandates: `is_move_constructible_v<E>` is `true` and `is_convertible_v<G, E>` is `true`.

Returns: `std::forward<G>(e)` if `has_value()` is `true`, `std::move(error())` otherwise.

Add a new section [expected.void.monadic] Monadic operations between [expected.void.obs] and [expected.void.eq]

1 `template <class F> constexpr auto and_then(F&& f) &;`

2 `template <class F> constexpr auto and_then(F&& f) const &;`

Let U be `remove_cvref_t<invoke_result_t<F>>`.

Constraints: `is_copy_constructible_v<E>` is `true`.

Mandates: U is a specialization of `expected` and `is_same_v<U::error_type, E>` is `true`.

Effects: Equivalent to:

```
if (has_value())
   return invoke(std::forward<F>(f));
else
   return U(unexpect, error());
```

1 `template <class F> constexpr auto and_then(F&& f) &&;`

2 `template <class F> constexpr auto and_then(F&& f) const &&;`

Let U be `remove_cvref_t<invoke_result_t<F>>`.

Constraints: `is_move_constructible_v<E>` is `true`.

Mandates: U is a specialization of `expected` and `is_same_v<U::error_type, E>` is `true`.

Effects: Equivalent to:

```
if (has_value())
   return invoke(std::forward<F>(f));
else
   return U(unexpect, std::move(error()));
```

1 `template <class F> constexpr auto or_else(F&& f) &;`

2 `template <class F> constexpr auto or_else(F&& f) const &;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Mandates: G is a specialization of `expected` and `is_same_v<G::value_type, T>` is `true`.

Effects: Equivalent to:

```
if (has_value())
    return G();
else
    return invoke(std::forward<F>(), error());
```

1 `template <class F> constexpr auto or_else(F&& f) &&;`

2 `template <class F> constexpr auto or_else(F&& f) const &&;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

Mandates: G is a specialization of `expected` and `is_same_v<G::value_type, T>` is `true`.

Effects: Equivalent to:

```
if (has_value())
   return G();
else
   return invoke(std::forward<F>(f), std::move(error()));
```

1 `template <class F> constexpr auto transform(F&& f) &;`

2 `template <class F> constexpr auto transform(F&& f) const &;`

Let U be `remove_cv_t<invoke_result_t<F>>`.

Mandates: U is a valid value type for `expected`. If `is_void_v<U>` is `false`, the declaration `U u(invoke(std::forward<F>(f)));` is well-formed.

Constraints: `is_copy_constructible_v<E>` is `true`.

Effects:

- If `has_value()` is `false`, returns `expected<U,E>(unexpect, error())`.
- Otherwise, if `is_void_v<U>` is `false`, returns an `expected<U, E>` object whose `has_val` member is `true` and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f))`.
- Otherwise, evaluates `invoke(std::forward<F>(f))` and then returns `expected<U,E>()`.

1 `template <class F> constexpr auto transform(F&& f) &&;`

2 `template <class F> constexpr auto transform(F&& f) const &&;`

Let U be `remove_cv_t<invoke_result_t<F>>`.

Mandates: U is a valid value type for `expected`. If `is_void_v<U>` is `false`, the declaration `U u(invoke(std::forward<F>(f)));` is well-formed.

Constraints: `is_move_constructible_v<E>` is `true`.

Effects:

- If `has_value()` is `false`, returns `expected<U,E>(unexpect, std::move(error()))`.
- Otherwise, if `is_void_v<U>` is `false`, returns an `expected<U, E>` object whose `has_val` member is `true` and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f))`.
- Otherwise, evaluates `invoke(std::forward<F>(f))` and then returns `expected<U,E>()`.

1 `template <class F> constexpr auto transform_error(F&& f) &;`

2 `template <class F> constexpr auto transform_error(F&& f) const &;`

Let G be `remove_cv_t<invoke_result_t<F, decltype(error())>>`.

Mandates: G is a valid value type for `expected` and the declaration `G g(invoke(std::forward<F>(f), error()));` is well-formed.

Returns: If `has_value()` is `true`, `expected<T,G>()`; otherwise, an `expected<T, G>` object whose `has_val` member is `false` and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), error())`.

1 `template <class F> constexpr auto transform_error(F&& f) &&;`

2 `template <class F> constexpr auto transform_error(F&& f) const &&;`

Let G be `remove_cv_t<invoke_result_t<F, decltype(std::move(error()))>>`.

Mandates: G is a valid value type for `expected` and the declaration `G g(invoke(std::forward<F>(f), std::move(error())));` is well-formed.

Returns: If `has_value()` is `true`, `expected<T,G>()`; otherwise an `expected<T, G>` object whose `has_val` member is `false` and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(error()))`.

Acknowledgements

- Thanks to Jonathan Wakely for implementation and wording feedback.
- Thanks to Tomasz Kaminski for extensive wording input and suggestions on `error_or`.
- Thanks to Sy Brand for the original work on optional and expected.
- Thanks to Broneck Kozicki for early feedback.
- Thanks to Arthur O'Dwyer for design and wording feedback.
- Thanks to Barry Revzin for design and wording feedback as well as shepherding in LEWG.

Revision History

Version	Date	Changes
0	2021-12-12	Initial Draft
1	2022-02-10	added `transform_or`
		allow `transform` on `expected<void, E>`
		updated status of P0323 to plenary approved

Version	Date	Changes
		improved examples
		added missing overloads for `or_else`
		`or_else` signature and wording fixes
		added design discussion of changing return types
		variety of wording fixes
2	2022-04-15	updates to reference Wakely implmentation and examples
		remove unneeded invoke calls from void overloads
		`expected.object.swap` instead of modifiers in synopsis
		change constraint wording to use `is_copy_constructible_v` form for consistency
		change Let U for move overloads to `decltype(std::move(...))`
		fix `transform_error` and `or_else` constraints to be T instead of E
		rename `transform_or` to `transform_error`
		remove the `is_same_v` constraint on `transform_error`
		change `transform_error` true case to return `expected<T,U>`
		remove note: U will implicitly model either copy or move constructible…
		change all the `if (*this)` to `if (has_value())` to match latest expected wording
		remove the `in_place` tag in `transform` overloads
		remove void from `invoke_results` for `expected<void,E> and_then` and `transform`.

Version	Date	Changes
		change error returns for `and_then~/~transform` to `return U(unexpect, error())`
		change Let U to Let G for `transform_error` and `or_else` for less confusion
3	2022-06-05	Add a design discussion of why deducing-this is not utilized
		Add a discussion of free function versus member functions
		mandate `transform` return type not be `unexpected`, `inplace_t`, etc
		design discussion and wording for `transform` to `expected<void, E>`
		design discussion of `error_or` functions
4	2022-06-15	Add an `error_or` function that mirrors `value_or` for basic case
		Update design section and intro with `error_or` information
		Incorporate wording suggestions from Tomasz K.
		Add link to Rishabh D. implementation
5	2022-09-20	fix dangling ', and' on several constraints clauses
		fix otherwise structures removing list and using semi-colons
		remove the stray 'i' in the `error_or` signatures
		change `is_convertible` to `is_convertible_v`
		fix context references for synopsis and elsewhere
		for transform overloads make `remove_cvref_t` `remove_cv_t`
		remove `std::move()` from lvalue `or_else`

Version	Date	Changes
		remove comment about optional update of feature macro since gcc has shipped
		add space to `const&` so it's `const &` everywhere
		define valid expected type and use in specification of some overloads
		for `transform_error` make `remove_cvref_t` be `remove_cv_t`
		remove braces from effect equivalent to clauses

References

1. [P0798] Sy Brand "Monadic Functions for std::optional" https://wg21.link/P0798
2. C++ draft [optional.nomadic] http://eel.is/c++draft/optional.monadic
3. [P0323] Vicente Botet, JF Bastien, Jonathan Wakely std::expected https://wg21.link/P0323
4. Sy Brand expected https://github.com/TartanLlama/expected
5. Sy Brand expected docs https://tl.tartanllama.xyz/en/latest/api/expected.html#tl-expected
6. Jonathan Wakely full implemenation https://github.com/jwakely/gcc/blob/expected/libstdc%2B%2B-v3/include/std/expected
7. Jonathan Wakely live demo https://godbolt.org/z/3a1j6d63a
8. Rust Result https://doc.rust-lang.org/std/result/enum.Result.html
9. More complete examples https://godbolt.org/z/va5fzx11f
10. Source for this proposal https://github.com/JeffGarland/wg21work/tree/main/monadic_expected